

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении курсовой работы
по дисциплине «Методы проектирования баз данных»
«Программирование и настройка СУБД»

№	Лабораторная	Дата сдачи
1	Представления	17.09.25
2	Триггеры	24.09.25
3	Роли	15.10.25
4	Процедуры и функции	22.10.25
5	Транзакции	29.10.25

Выполнила студентка группы
№5130201/30101

Проверил

Лаишевкина А.М.

Попов С.Г.

«10»

2025г.

Санкт-Петербург, 2025

Реферат

В данной работе рассмотрены нереляционные возможности СУБД, такие как триггеры, хранимые процедуры и функции. В ходе работы были созданы триггеры для автоматического обновления статистики, а также функция и процедура для работы с данными. Помимо этого были также рассмотрены создание пользователей и выдача им различных прав. Для этого была использована система управления базами данных MySQL. В результате имеющаяся база данных была автоматизирована, что можно применять для более удобного использования и упрощения работы с информацией.

Содержание

Введение	4
1 Постановка задачи	5
2 Основная часть	6
2.1 Лабораторная работа №1. Представления	6
2.1.1 Создание представления	6
2.1.2 Запросы	6
2.1.3 Обновляемость представления	8
2.2 Лабораторная работа №2. Триггеры	9
2.2.1 Создание таблицы	9
2.2.2 Создание триггеров	9
2.3 Лабораторная работа №3. Права пользователей	13
2.3.1 Создание пользователей	13
2.4 Лабораторная работа №4. Функции и процедуры	15
2.4.1 Создание функции	15
2.4.2 Создание процедуры	16
2.5 Лабораторная работа №5. Транзакции	18
2.5.1 Постановка задачи	18
2.5.2 Выполнение задания	18
Заключение	21
Список использованных источников	22
Приложение А. Исходный код создания функции	23
Приложение Б. Исходный код создания процедуры	24

Введение

Объем данных в мире увеличивается с каждым годом, поэтому системы управления базами данных становятся неотъемлемым инструментом для эффективного хранения и управления информацией. Большую роль в этом играют нереляционные возможности СУБД. Они позволяют автоматизировать бизнес-логику, контролировать целостность данных и управлять доступом пользователей на уровне самой базы данных.

В данной работе рассматриваются нереляционные возможности СУБД MySQL: представления, триггеры, хранимые процедуры и функции, управление правами пользователей, а также транзакции с различными уровнями изоляции. Эти инструменты позволяют переложить часть логики приложения на уровень базы данных, что способствует повышению производительности и усилению безопасности данных.

Целью данной курсовой работы является изучение и практическое применение нереляционных возможностей СУБД для автоматизации и оптимизации работы с данными в контексте базы данных ивент-агентства.

Результатом работы является автоматизированная база данных, в которой за счёт использования встроенных средств СУБД обеспечены целостность данных, контролируемый многопользовательский доступ и реализована часть бизнес-правил.

1 Постановка задачи

Задачи данной работы:

1. Создать представление на основе выполненного ранее запроса, а также выполнить к нему 2 запроса.
2. Реализовать триггеры для автоматического поддержания актуальности данных в одной из таблиц при изменениях в других таблицах.
3. Создать 2-х пользователей и выдать им права на разные таблицы.
4. Написать функцию и процедуру.
5. Реализовать сценарий взаимодействия пользователей с базой данных с помощью транзакций с целью показать наличие определенной аномалии при определенном уровне изоляции.

2 Основная часть

2.1 Лабораторная работа №1. Представления

Задачи к лабораторной работе №1:

1. Создать представления на основе выполненного ранее запроса с агрегатными функциями.
2. Выполнить простой запрос к этому представлению.
3. Выполнить сложный запрос с присоединением к представлению других таблиц.
4. Проверить обновляемость представления.

2.1.1 Создание представления

Запрос, на основе которого было построено представление: для каждого агентства посчитать число поставщиков. Код создания представления представлен в [листинге 1](#).

Листинг 1. Представление `agency_suppliers_count`

```
1 CREATE VIEW agency_suppliers_count AS
2 SELECT
3     a.agency_id,
4     a.name AS agency_name,
5     COUNT(DISTINCT ls.supplier_id) AS suppliers_count
6 FROM
7     agency a
8 LEFT JOIN
9     list_of_suppliers ls ON a.agency_id = ls.agency_id
10 GROUP BY
11     a.agency_id, a.name
12 ORDER BY
13     a.agency_id;
```

2.1.2 Запросы

Был составлен простой запрос, который выводит только агентства, у которых более 11 поставщиков. В результирующей таблице было выведено 7 строк. Код запроса представлен в [листинге 2](#). Результирующая таблица представлена на рис. [Рис. 1](#).

Листинг 2. Простой запрос к представлению

```
1 SELECT agency_id, agency_name, suppliers_count
2 FROM agency_suppliers_count
```

```

3 WHERE suppliers_count > 11
4 ORDER BY suppliers_count DESC;

```

	agency_id	agency_name	suppliers_count
►	7	Williams-Mills	15
	2	Clayton PLC	14
	13	Oneill Group	14
	15	Brown Group	14
	9	Kane Ltd	13
	11	Robinson-Jones	12
	12	Myers and Sons	12

Рис. 1. Результат простого запроса

Далее был составлен сложный запрос, в котором осуществляется присоединение таблиц manager и event. Запрос выводит все агентства, количество их поставщиков, а также количество мероприятий, проведенных данным агентством. В результирующей таблице было выведено 15 строк. Код запроса представлен в [листинге 3](#). Результирующая таблица представлена на [Рис. 2](#).

Листинг 3. Сложный запрос к представлению

```

1 SELECT
2     asc_view.agency_id,
3     asc_view.agency_name,
4     asc_view.suppliers_count,
5     COUNT(e.event_id) AS events_count
6 FROM
7     agency_suppliers_count asc_view
8 LEFT JOIN
9     manager m ON asc_view.agency_id = m.agency_id
10 LEFT JOIN
11     event e ON m.manager_id = e.manager_id
12 GROUP BY
13     asc_view.agency_id, asc_view.agency_name, asc_view.suppliers_count
14 ORDER BY
15     events_count DESC;

```

	agency_id	agency_name	suppliers_count	events_count
►	14	Shepard, Silva and Douglas	10	1236
	4	Lewis Ltd	10	1200
	12	Myers and Sons	12	1104
	13	Oneill Group	14	1068
	7	Williams-Mills	15	1066
	3	Horton, Walters and Webb	10	1047
	11	Robinson-Jones	12	1037
	9	Kane Ltd	13	931
	10	Douglas and Sons	10	916
	8	Dixon-Martinez	10	915
	6	Weaver-Nixon	11	911
	5	Acosta and Sons	11	896
	1	Lawson-Watkins	11	894
	15	Brown Group	14	893
	2	Clayton PLC	14	886

Рис. 2. Результат сложного запроса

2.1.3 Обновляемость представления

Представление является обновляемым, если выполнены два условия: оно не содержит вычисляемых полей, агрегатных функций и операций объединения, оно основано на одной таблице и содержит все ее ключевые поля.

В данном представлении есть агрегатная функция count, которая вычисляет количество поставщиков для каждого агентства, а также group by, который не гарантирует отношение 1 к 1. Таким образом, представление является необновляемым.

Код для проверки необновляемости приведен в [листинге 4](#).

Листинг 4. Проверка обновляемости представления

```

1 UPDATE agency_suppliers_count
2 SET suppliers_count = 10
3 WHERE agency_id = 1;
4
5 INSERT INTO agency_suppliers_count (agency_id, agency_name,
6   suppliers_count)
7 VALUES (100, 'New Agency', 5);

```


2.2 Лабораторная работа №2. Триггеры

Задачи к лабораторной работе №2:

1. Создать таблицу для учета количества мероприятий в каждом из мест.
2. Реализовать триггеры для автоматического поддержания актуальности данных при изменениях в таблицах мест и мероприятий.
3. Проверить обновляемость таблицы.

2.2.1 Создание таблицы

Была создана таблица для учета количества мероприятий в каждом из мест. Схема базы данных после добавления таблицы представлена на [Рис. 3](#). Код создания и заполнения таблицы для уже существующих мест и мероприятий приведен в [листинге 5](#).

Листинг 5. Таблица place_event_stats

```
1 CREATE TABLE place_event_stats (  
2     place_id INT NOT NULL,  
3     address VARCHAR(50),  
4     events_count INT DEFAULT 0,  
5     PRIMARY KEY (place_id)  
6 );
```

2.2.2 Создание триггеров

Затем были созданы триггеры для insert, delete и update для таблицы event, а также триггеры для insert и delete для таблицы place. Код триггеров приведен в [листинге 6](#) и [листинге 7](#)^[1].

Триггер after_place_insert при добавлении нового места автоматически создает запись в таблице place_event_stats с нулевым количеством мероприятий.

Триггер after_place_delete при удалении места автоматически удаляет соответствующую запись из таблицы place_event_stats.

Листинг 6. Триггеры для таблицы place

```
1 CREATE TRIGGER after_place_insert  
2 AFTER INSERT ON place  
3 FOR EACH ROW  
4     INSERT INTO place_event_stats (place_id, address, events_count)  
5     VALUES (NEW.place_id, NEW.address, 0);  
6 CREATE TRIGGER after_place_delete
```

```

7 AFTER DELETE ON place
8 FOR EACH ROW
9     DELETE FROM place_event_stats WHERE place_id = OLD.place_id;

```

Триггер after_event_insert при добавлении мероприятия увеличивает счетчик мероприятий для указанного места.

Триггер after_event_delete при удалении мероприятия уменьшает счетчик мероприятий для связанного места.

Триггер after_event_update при изменении места у мероприятия корректирует счетчики обоих мест (старого и нового).

Листинг 7. Триггеры для таблицы event

```

1 CREATE TRIGGER after_event_insert
2 AFTER INSERT ON event
3 FOR EACH ROW
4     UPDATE place_event_stats SET events_count = events_count + 1 WHERE
      place_id = NEW.place_id;
5
6 CREATE TRIGGER after_event_delete
7 AFTER DELETE ON event
8 FOR EACH ROW
9     UPDATE place_event_stats SET events_count = events_count - 1 WHERE
      place_id = OLD.place_id;
10
11 DELIMITER //
12 CREATE TRIGGER after_event_update
13 AFTER UPDATE ON event
14 FOR EACH ROW
15 BEGIN
16     IF OLD.place_id IS NOT NULL THEN
17         UPDATE place_event_stats SET events_count = events_count - 1
18         WHERE place_id = OLD.place_id;
19     END IF;
20     IF NEW.place_id IS NOT NULL THEN
21         UPDATE place_event_stats SET events_count = events_count + 1
22         WHERE place_id = NEW.place_id;
23     END IF;
24 END //
25 DELIMITER ;

```

Результаты выполнения различных действий, влияющих на таблицы посредством триггеров, представлены в [таблице 1](#).

Таблица 1. Результаты запросов для пользователей с разными правами

№	Действие
1	Добавление нового места в таблицу <code>place</code> . В таблице <code>place_event_stats</code> создается запись с нулевым количеством мероприятий.
2	Удаление места из таблицы <code>place</code> . В таблице <code>place_event_stats</code> удаляется соответствующая запись.
3	Добавление мероприятия в таблицу <code>event</code> . Счетчик мероприятий для указанного места в таблице <code>place_event_stats</code> увеличивается на 1.
4	Удаление мероприятия из таблицы <code>event</code> . Счетчик мероприятий для указанного места в таблице <code>place_event_stats</code> уменьшается на 1.
5	Изменение места у мероприятия в таблице <code>event</code> . Счетчик мероприятий для нового места в таблице <code>place_event_stats</code> увеличивается на 1, а счетчик для старого места - уменьшается на 1.

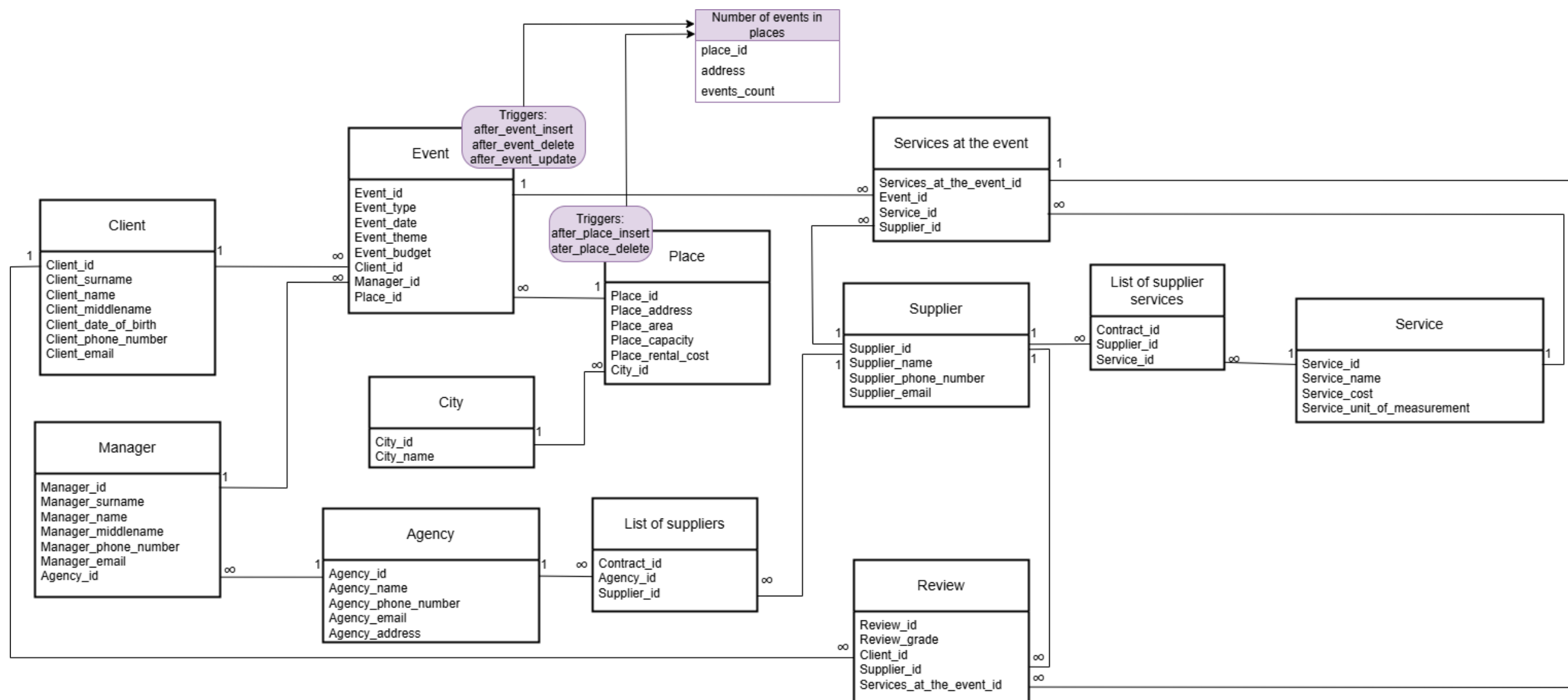


Рис. 3. Схема базы данных

2.3 Лабораторная работа №3. Права пользователей

Задачи к лабораторной работе №3:

1. Создать 2 пользователей.
2. Выдать пользователям права: для user1 только право на просмотр таблицы place_event_stats, для user2 права на просмотр и изменение таблиц place и event.
3. Проверить работу различных запросов от имени пользователей.

2.3.1 Создание пользователей

Было создано 2 новых пользователя user1 и user2, каждому из которых был предоставлен доступ к базе данных event_agency и выданы права на различные действия с различными таблицами[2]. User1 были предоставлены права только на просмотр таблицы для учета количества мероприятий в каждом из мест place_event_stats, а user2 – права на просмотр таблиц place, event, client, manager, а также изменение таблиц place и event. Таким образом, user2 может изменять таблицы, на которые повешены триггеры, которые в свою очередь изменят единственную таблицу, доступную для просмотра пользователем user1. Код выдачи прав представлен в [листинге 8](#).

Листинг 8. Права пользователей

```
1 GRANT SELECT ON event_agency.place_event_stats TO 'user1'@'localhost';
2
3 GRANT SELECT, INSERT, UPDATE, DELETE ON event_agency.place TO 'user2'@'
  localhost';
4 GRANT SELECT, INSERT, UPDATE, DELETE ON event_agency.event TO 'user2'@'
  localhost';
5 GRANT SELECT ON event_agency.client TO 'user2'@'localhost';
6 GRANT SELECT ON event_agency.manager TO 'user2'@'localhost';
7
8 FLUSH PRIVILEGES;
```

Результаты выполнения кода и, соответственно, предоставленные пользователям права представлены на [Рис. 4](#) и [Рис. 5](#). Результаты выполнения пользователями различных действий представлены в [таблице 2](#).

```
mysql> SHOW GRANTS FOR 'user1'@'localhost';
+-----+
| Grants for user1@localhost |
+-----+
| GRANT USAGE ON *.* TO 'user1'@'localhost' |
| GRANT SELECT ON 'event_agency'.'place_event_stats' TO 'user1'@'localhost' |
+-----+
```

Рис. 4. Права user1

```
mysql> SHOW GRANTS FOR 'user2'@'localhost';
+-----+
| Grants for user2@localhost |
+-----+
| GRANT USAGE ON *.* TO 'user2'@'localhost' |
| GRANT SELECT ON 'event_agency'.'client' TO 'user2'@'localhost' |
| GRANT SELECT, INSERT, UPDATE, DELETE ON 'event_agency'.'event' TO 'user2'@'localhost' |
| GRANT SELECT ON 'event_agency'.'manager' TO 'user2'@'localhost' |
| GRANT SELECT, INSERT, UPDATE, DELETE ON 'event_agency'.'place' TO 'user2'@'localhost' |
+-----+
```

Рис. 5. Права user2

Таблица 2. Результаты запросов для пользователей с разными правами

№	user1	user2
1	SELECT * FROM place_event_stats;	
	Таблица place_event_stats	Ошибка доступа
2	SELECT * FROM place;	
	Ошибка доступа	Таблица place
3	INSERT INTO place VALUES (56, 'Test Venue', 100, 50, 5000, 3);	
	Ошибка доступа	Выполненный запрос
4	INSERT INTO event VALUES (15001, 'party');	
	Ошибка доступа	Выполненный запрос
5	UPDATE place SET address = 'New Address' WHERE place_id = 11;	
	Ошибка доступа	Выполненный запрос
6	DELETE FROM event WHERE event_id = 15001;	
	Ошибка доступа	Выполненный запрос

2.4 Лабораторная работа №4. Функции и процедуры

Задачи к лабораторной работе №4:

1. Реализовать функцию, которая принимает на вход имя, фамилию и отчество и возвращает строку с фамилией и инициалами.
2. Написать процедуру, которая принимает на вход названия места и города, проверяет их наличие в таблицах и реализует различную логику при определенных условиях.

2.4.1 Создание функции

Функция `get_full_name` принимает на вход 3 поля типа `VARCHAR`: фамилия, имя и отчество, а на выход передает одно поле типа `VARCHAR` - фамилию и инициалы. Код функции представлен в [Приложении А](#).

Чтобы проверить работу функции был написан запрос, который с помощью функции выводит фио менеджеров из таблицы `manager`. Код запроса представлен в [листинге 9](#). Результат работы функции представлен на [Рис. 6](#).

Листинг 9. Применение функции

```
1 SELECT
2     manager_id ,
3     surname ,
4     name ,
5     middlename ,
6     get_full_name(surname , name , middlename) as fullname
7 FROM manager;
```

	manager_id	surname	name	middlename	fullname
►	1	Newman	Kyle	NULL	Newman K.
	2	Carroll	Brian	Young	Carroll B. Y.
	3	Silva	Jeremy	NULL	Silva J.
	4	Anderson	Michael	Hernandez	Anderson M. H.
	5	Villa	Dale	Bernard	Villa D. B.
	6	Greene	Dennis	Ramsey	Greene D. R.
	7	Clark	Michael	Ware	Clark M. W.
	8	Pacheco	Joann	Day	Pacheco J. D.
	9	Greene	Justin	NULL	Greene J.
	10	Aguirre	William	NULL	Aguirre W.
	11	Stevens	Cory	Davenport	Stevens C. D.
	12	Reyes	Diana	Rogers	Reyes D. R.
	13	Perkins	Sarah	Wallace	Perkins S. W.
	14	Jarvis	Kather...	Dawson	Jarvis K. D.
	15	Flores	Marisa	NULL	Flores M.

Рис. 6. Результат работы функции

2.4.2 Создание процедуры

Процедура `manage_place_city` принимает на вход два поля типа `VARCHAR`: название места и название города. Далее процедура вычисляет `id` места и города, проверяя, есть ли такие значения в соответствующих таблицах. После этого имеют место следующие сценарии работы процедуры:

1. Если и город, и место существуют, и при этом они связаны, т.е. данное место находится в данном городе, то процедура выводит сообщение об этом.
2. Если и место, и город существуют, но не связаны, то процедура создает новую запись в таблице `place` с данным местом и связывает ее с городом.
3. Если город существует, а место нет, процедура точно так же создает новую запись в таблице `place` и связывает с заданным городом.
4. Если место существует, а города нет, то процедура выводит сообщение об этом.
5. Если не существует ни места, ни города, то процедура создает две записи - одну в таблице `place`, другую в таблице `city`, - и связывает их.

Блок-схема алгоритма работы процедуры представлена на [Рис. 7](#). Код процедуры представлен в [Приложении Б](#).

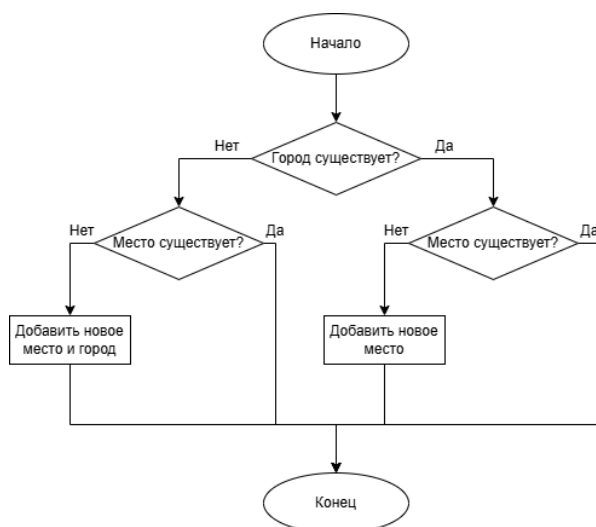


Рис. 7. Блок-схема алгоритма работы процедуры

Результат работы процедуры при вводе уже существующих и связанных друг с другом места и города представлен на [Рис. 8](#).

```
mysql> CALL manage_place_city('5349 Mayer Prairie', 'Rosestad');
+-----+
| result |
+-----+
| Everything is fine! This place already exists in this city. |
+-----+
1 row in set (0.01 sec)
```

Рис. 8. Результат работы процедуры

Результат работы процедуры при вводе уже существующих и несвязанных друг с другом места и города представлен на [Рис. 9](#).

```
mysql> CALL manage_place_city('4005 Susan Fords', 'Shannonstad');
+-----+
| result |
+-----+
| Place is added to this city. |
+-----+
1 row in set (0.01 sec)
```

Рис. 9. Результат работы процедуры

Результат работы процедуры при вводе несуществующего места и существующего города представлен на [Рис. 10](#).

```
mysql> CALL manage_place_city('8888 Picodilli Park', 'Lyonsbury');
+-----+
| result |
+-----+
| Place is added to this city. |
+-----+
1 row in set (0.02 sec)
```

Рис. 10. Результат работы процедуры

Результат работы процедуры при вводе существующего места и несуществующего города представлен на [Рис. 11](#).

```
mysql> CALL manage_place_city('5349 Mayer Prairie', 'Chicago');
+-----+
| result |
+-----+
| City does not exists but the place does. |
+-----+
1 row in set (0.00 sec)
```

Рис. 11. Результат работы процедуры

Результат работы процедуры при вводе несуществующих места и города представлен на [Рис. 12](#).

```
mysql> CALL manage_place_city('3131 Marysind Street', 'Chicago');
+-----+
| result |
+-----+
| New place and new city are added. |
+-----+
1 row in set (0.02 sec)
```

Рис. 12. Результат работы процедуры

2.5 Лабораторная работа №5. Транзакции

2.5.1 Постановка задачи

Реализовать сценарий взаимодействия пользователей с базой данных с помощью транзакций с целью показать наличие определенной аномалии при определенном уровне изоляции.

2.5.2 Выполнение задания

Для данной работы были использованы созданные ранее пользователи user1 и user2. Для выполнения работы были выбраны уровень изоляции транзакций repeatable read и аномалия данных - потерянные изменения[3]. Далее от имени пользователей были одновременно запущены 2 транзакции. Сценарий их взаимодействия с базой данных представлен в [таблице 3](#).

Таблица 3. Транзакции двух пользователей

№	user1	user2
1	START TRANSACTION; SELECT event_id, budget FROM event WHERE event_id = 100; user1 начинает транзакцию, для user1 budget = 338802	—
2	—	START TRANSACTION; SELECT event_id, budget FROM event WHERE event_id = 100; user2 начинает транзакцию, для user2 budget = 338802
3	UPDATE event SET budget = budget + 100000 WHERE event_id = 100; user1 обновляет значение budget, для user1 budget = 438802	—
4	—	SELECT event_id, budget FROM event WHERE event_id = 100; для user2 все еще budget = 338802, операции не выполняются, потому что ожидают коммита от user1
5	COMMIT; user1 коммитит изменения	—

№	user1	user2
6	—	UPDATE event SET budget = budget + 200000 WHERE event_id = 100; COMMIT; user2 обновляет значение budget, для user2 budget = 638802, затем user2 коммитит изменения
7	SELECT event_id, budget FROM event WHERE event_id = 100; для user1 budget = 638802	SELECT event_id, budget FROM event WHERE event_id = 100; для user2 budget = 638802

При уровне изоляции repeatable read на шагах 1 и 2 оба пользователя начинают транзакции и видят исходное значение бюджета n. На шаге 3 user1 выполняет update, увеличивая бюджет до n+100000, но это изменение видно только ему. На шаге 4 user2 продолжает видеть исходное значение n благодаря механизму снимков данных. После коммита user1 на шаге 5 данные физически обновляются до n+100000. На шаге 6 при выполнении update user2 видит актуальное значение n+100000 (поскольку операция записи всегда работает с текущим состоянием данных) и увеличивает его до n+300000. В результате на шаге 7 оба пользователя видят корректное итоговое значение n+300000, что демонстрирует отсутствие аномалии потерянных изменений - механизм блокировок в сочетании обеспечивает сохранение всех изменений, и оба обновления успешно применяются к данным.

Результаты выполнения шагов транзакций представлены на [Рис. 12-17](#)

```
mysql> SELECT event_id, budget FROM event WHERE event_id = 100;
+-----+-----+
| event_id | budget |
+-----+-----+
|      100 | 338802 |
+-----+-----+
1 row in set (0.01 sec)
```

Рис. 13. Исходное значение budget для user1 и user2

```
mysql> UPDATE event SET budget = budget + 100000 WHERE event_id = 100;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT event_id, budget FROM event WHERE event_id = 100;
+-----+-----+
| event_id | budget |
+-----+-----+
|      100 | 438802 |
+-----+-----+
1 row in set (0.00 sec)
```

Рис. 14. Обновление значения budget для user1

```
mysql> SELECT event_id, budget FROM event WHERE event_id = 100;
+-----+-----+
| event_id | budget |
+-----+-----+
|      100 | 338802 |
+-----+-----+
1 row in set (0.00 sec)
```

Рис. 15. Текущее значение budget для user2

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Рис. 16. Коммит user1

```
mysql> UPDATE event SET budget = budget + 200000 WHERE event_id = 100;
Query OK, 1 row affected (6.99 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

Рис. 17. Обновление значения budget для user2

```
mysql> SELECT event_id, budget FROM event WHERE event_id = 100;
+-----+-----+
| event_id | budget |
+-----+-----+
|      100 | 638802 |
+-----+-----+
1 row in set (0.00 sec)
```

Рис. 18. Значение budget для user1 и user2 после завершения транзакций

Заключение

В первой лабораторной работе было создано представление `agency_suppliers_count` на основе агрегирующего запроса. К представлению были выполнены два запроса. Была доказана необновляемость данного представления из-за наличия агрегатной функции и группировки.

Во второй работе была создана служебная таблица `place_event_stats` для учета количества мероприятий в каждом месте. Для автоматического поддержания её актуальности были разработаны и протестированы шесть триггеров.

В третьей лабораторной работе были созданы два пользователя (`user1` и `user2`) и настроена система прав доступа.

В четвёртой работе были созданы и протестированы хранимая функция и процедура. Функция `get_full_name` форматирует ФИО в фамилию с инициалами. Процедура `manage_place_city` реализует логику добавления места и города, обрабатывая пять возможных сценариев их наличия или отсутствия в базе данных.

В пятой лабораторной работе на примере пользователей `user1` и `user2` был исследован механизм транзакций на уровне изоляции `REPEATABLE READ`. С помощью сценария было продемонстрировано, как система блокировок и снимков данных предотвращает аномалию «потерянных обновлений».

Все работы выполнялись с использованием системы управления базами данных MySQL. В качестве основного инструмента разработки и тестирования SQL-кода использовался MySQL Command Line Client. Для визуализации структуры базы данных применялись средства построения диаграмм.

Список использованных источников

1. Trigger Syntax and Examples / [Электронный ресурс] // MySQL : [сайт]. — URL: <https://dev.mysql.com/doc/refman/8.4/en/trigger-syntax.html> (дата обращения: 03.12.2025).
2. Создание нового пользователя и раздача прав в MySQL / [Электронный ресурс] // timeweb.cloud : [сайт]. — URL: <https://timeweb.cloud/tutorials/mysql/kak-sozdat-novogo-polzovatelya-i-razdat-prava-v-mysql> (дата обращения: 03.12.2025).
3. Уровни изоляции транзакций в БД / [Электронный ресурс] // Habr.com : [сайт]. — URL: <https://habr.com/ru/articles/845522/> (дата обращения: 03.12.2025).

Приложение А. Исходный код создания функции

Листинг 10. Создание функции

```
1 DELIMITER //
```

```
2 CREATE FUNCTION get_full_name(  
3     surname VARCHAR(50),  
4     name VARCHAR(50),  
5     middlename VARCHAR(50)  
6 )  
7 RETURNS VARCHAR(100) DETERMINISTIC  
8 BEGIN  
9     RETURN CONCAT(  
10         surname, ' ',  
11         UPPER(SUBSTRING(name, 1, 1)), '.',  
12         IF(middlename IS NOT NULL, CONCAT(' ', UPPER(SUBSTRING(  
13             middlename, 1, 1)), '.'), '')  
14     );  
15 END//  
16 DELIMITER ;
```

Приложение Б. Исходный код создания процедуры

Листинг 11. Создание процедуры

```
1 DELIMITER //
2 CREATE PROCEDURE manage_place_city(
3     IN input_place_name VARCHAR(100),
4     IN input_city_name VARCHAR(50)
5 )
6 BEGIN
7     DECLARE city_id_val INT DEFAULT NULL;
8     DECLARE place_exists_anywhere BOOLEAN DEFAULT FALSE;
9     DECLARE place_exists_in_city BOOLEAN DEFAULT FALSE;
10
11     SELECT city_id INTO city_id_val
12     FROM city
13     WHERE city_name = input_city_name
14     LIMIT 1;
15
16     SELECT EXISTS(SELECT 1 FROM place WHERE address = input_place_name)
17     INTO place_exists_anywhere;
18
19     IF city_id_val IS NOT NULL THEN
20         SELECT EXISTS(SELECT 1 FROM place WHERE address =
21             input_place_name AND city_id = city_id_val)
22         INTO place_exists_in_city;
23
24         IF place_exists_in_city THEN
25             SELECT 'Everything is fine! This place already exists in
26                 this city.' AS result;
27         ELSE
28             INSERT INTO place (address, area, capacity, rental_cost,
29                 city_id)
30             VALUES (input_place_name, 100, 50, 10000, city_id_val);
31             SELECT 'Place is added to this city.' AS result;
32         END IF;
33     ELSE
34         IF place_exists_anywhere THEN
35             SELECT 'City does not exist, but the place already exists
36                 elsewhere.' AS result;
37         ELSE
38             INSERT INTO city (city_name) VALUES (input_city_name);
39             SET city_id_val = LAST_INSERT_ID();
40             INSERT INTO place (address, area, capacity, rental_cost,
41                 city_id)
42             VALUES (input_place_name, 100, 50, 10000, city_id_val);
43             SELECT 'New place and new city are added.' AS result;
44         END IF;
45     END IF;
46 END//
47 DELIMITER ;
```